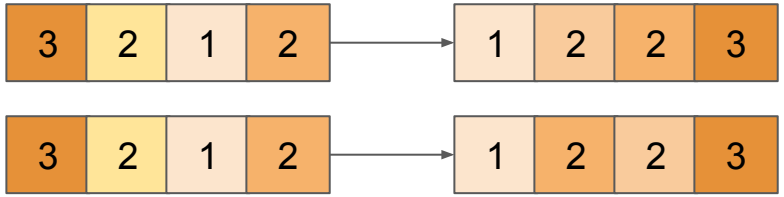
Algoritmos de Ordenamiento

Aspectos a analizar

- Costo Amortizado
- Consideraciones de Mejor y Peor Caso
- Estabilidad:



Estable

No-Estable

- Espacio Adicional requerido (¿in-place?)
- Tipo de Algoritmo (Iterativo / Recursivo)

Bubble sort

6 5 3 1 8 7 2 4

```
for (cont=1; cont<=n-1; cont++)
    for (j=0; j<n-cont; j++)
        si a[j]>a[j+1]
            a[j] <--> a[j+1]
```

Costo Amort: O(n^2)

Selection sort

```
for (i=0; i<n-1; i++)
    minimo=i
    for (j=i+1; j<n; j++)
        si a[j]<a[minimo] minimo=j
    a[i] <--> a[minimo]
```

	8
	5
	2
	6
	9
	3
	1
	4
	0
	7

Costo Amort: O(n^2)

Insertion sort

```
for (i=1; i<n; i++)
    X=a[i]
    for (j=i-1; j>=0 && a[j]>X; j=j-1)
        a[j+1]=a[j]
    a[j+1]=X
```

6 5 3 1 8 7 2 4

- Variantes:
  - Binary search insertion sort
  - Shell sort

Costo Amort: O(n^2)
Mejor Caso: O(n)

Variante de Insertion sort:Shell sort

```
Para cada Rango en {5,3,1}
    for (i=Rango; i<n; i++)
        X=a[i]
        for (j=i-Rango; j>=0 && a[j]>X; j=j-Rango)
                a[j+Rango]=a[j]
        a[j+Rango]=X
```

	V[0]	V[1]	V[2]	V[3]	V[4]	V[5]	V[6]	V[7]	V[8]	V[9]	V[10]	V[11]
Entrada	62	83	18	53	07	17	95	86	47	69	25	28
Pos sort (5)	17	28	18	47	07	25	83	86	53	69	62	95
Pos sort (3)	17	07	18	47	28	25	69	62	53	83	86	95
Pos sort (1)	07	17	18	25	28	47	53	62	69	83	86	95

Merge sort

6 5 3 1 8 7 2 4

```
MergeSort(A[],i,s)
    Si i==s return;
    Medio=(i+s)/2
    MergeSort(A[],i,Medio)
    MergeSort(A[],Medio+1,s])
    Merge(A[],i,Medio,s);
```

Costo Amort: O(n*log(n))

Heap sort

HeapSort(A[])

Heap.Create()

Heap.Heapify(A[]) [O(n)]

Extraigo n veces la raíz [n veces O(Logn)]

Costo Amort: O(n*log(n))

Quick sort

6 5 3 1 8 7 2 4

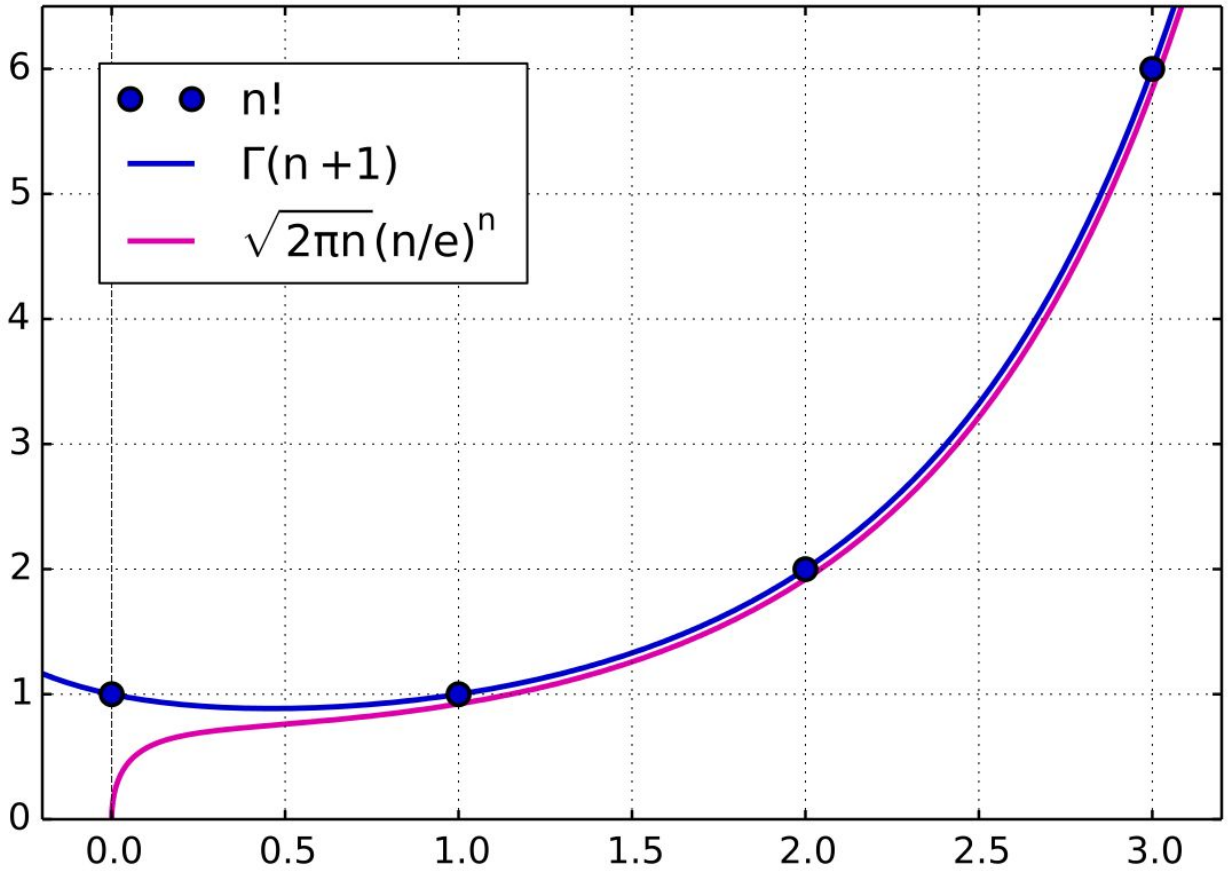
```
QuickSort(A[],i,s)
    if (i<h)
        p=Dividir(A[],i,s)
        QuickSort(A[],i,p-1)
        QuickSort(A[],p+1,s)
```

Costo Amort: O(n*log(n))
Peor Caso: O(n^2)

Teorema: “No se puede ordenar por Comparación en menos de O(n*log(n)) “

- Analizar àrbol posibilidades de Ordenamiento
  - Posibles Ordenamientos (hojas): N!
  - Altura del Árbol= Log(Hojas)=Log(N!)
- Aproximación de Stirling

Aproximación de Stirling



Teorema: “No se puede ordenar por Comparación en menos de O(n*log(n)) “

Altura del Árbol= Log(Hojas) =

Log(N!) =
Log(sqrt(2*pi*n) * (n/e)^n) =
Log(sqrt(2*pi*n)) + Log((n/e)^n) =
Log(sqrt(2*pi*n)) + n* Log(n/e)) → O (n * Log(n))

Algoritmos de Ordenamiento <u>sin comparar</u>

- **Counting Sort**
  - Algoritmo:
    - Utilizar un arreglo auxiliar para contar las apariciones de cada elemento.
    - Recorro el arreglo y cuento las apariciones de cada valor.
    - Reconstruyo el arreglo en base al conteo. NUNCA comparé!
  - Solo sirve para “espacios de valores” pequeños
  - No es in-place

Costo Amort: O(n)

Algoritmos de Ordenamiento <u>sin comparar</u>

- **Radix Sort**
  - Algoritmo:
    - Utiliza listas para ordenar de 1 dígito
  - NUNCA comparé los valores!
  - No es in-place

Costo Amort: O(kn)

Algoritmos de Ordenamiento <u>sin comparar</u>

- **Bucket Sort**
  - Algoritmo:
    i. Crear k Buckets mutuamente excluyentes (vacíos)
    ii. Para cada elemento…
        - Colocarlo en el bucket correspondiente
    iii. Ordenar individualmente cada bucket
    iv. Devolver los elementos de cada bucket concatenados por orden

  - NUNCA comparé los valores!

Costo Amort: O(kn)

Algoritmos Híbridos

- **Introspective sort (intro-sort)**
  - Algoritmo:
    i. Quick sort (hasta elementos ~ log(N))
    ii. Heap sort (hasta elementos ~ k)
    iii. Insertion sort

- Usado por C++ para ordenamiento de STL
- In-place
- No-Estable

Algoritmos Híbridos

- **Tim sort**
  - Se base en Merge sort e Insertion sort
  - Idea:
    i. Busca secuencias de al menos k elementos ya ordenados
    ii. Hace un merge de esas secuencias

- Usado por Java y Python
- no In-place (*)
- Estable